

1. Aim

To automate the deployment of a web application using Jenkins by configuring build triggers, post-build actions and deployment steps in a declarative pipeline, so that every code push is tested, built and deployed to a server without manual intervention.

2. Objectives

- Install and configure Jenkins on an Ubuntu server.
- Create a Pipeline job that pulls source code from a Git repository.
- Configure build triggers: GitHub webhook, Poll SCM and scheduled build.
- Automate test, build and deployment stages using a Jenkinsfile.
- Configure post-build actions: archiving artifacts, e-mail notification and workspace cleanup.
- Verify the deployed application through a health-check.

3. Tools and Technologies

Tool	Version	Purpose
Ubuntu Server	22.04 LTS	Host OS for Jenkins and target server
Jenkins	2.462.x (LTS)	Automation server
Git / GitHub	2.34 / Cloud	Source code management and webhooks
Node.js and npm	18.x	Sample application runtime
PM2	5.x	Process manager on the deployment server
OpenJDK	17	Required by Jenkins

4. Theory

Continuous Integration (CI) is the practice of merging code changes frequently and validating each change with an automated build and test run. **Continuous Deployment (CD)** extends this by automatically releasing every change that passes all stages to a target environment.

Jenkins is an open-source automation server. Work is described as a *Pipeline* in a text file called **Jenkinsfile** stored with the source code. A pipeline is made up of *stages*, and stages contain *steps*. The **post** section runs after the stages finish and is used for notifications and cleanup.

Build triggers used

Trigger	How it works
GitHub hook trigger	GitHub sends an HTTP POST to Jenkins on every push, starting the build instantly.
Poll SCM	Jenkins checks the repository on a cron schedule and builds only if new commits exist.
Build periodically	Runs on a fixed cron schedule regardless of changes (e.g., nightly build).

5. Implementation Steps

Step 1: Install Java and Jenkins

Jenkins runs on Java. Install OpenJDK 17, add the Jenkins repository key and install the package.

```
ubuntu@jenkins-server: ~ $
```

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y fontconfig openjdk-17-jre
java -version

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

Output

```
openjdk version "17.0.12" 2024-07-16
OpenJDK Runtime Environment (build 17.0.12+7-Ubuntu-1ubuntu222.04)

* jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/lib/systemd/system/jenkins.service; enabled)
   Active: active (running) since Mon 2026-09-28 10:05:11 UTC; 12s ago
   Main PID: 3412 (java)
```

Step 2: Unlock Jenkins and install plugins

Open **http://<server-ip>:8080**, read the initial admin password with the command below, choose **Install suggested plugins**, then create the admin user. Also install these plugins from *Manage Jenkins > Plugins*: **Pipeline, Git, GitHub, SSH Agent, Email Extension, JUnit** and **Workspace Cleanup**.

```
ubuntu@jenkins-server: ~ $
```

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

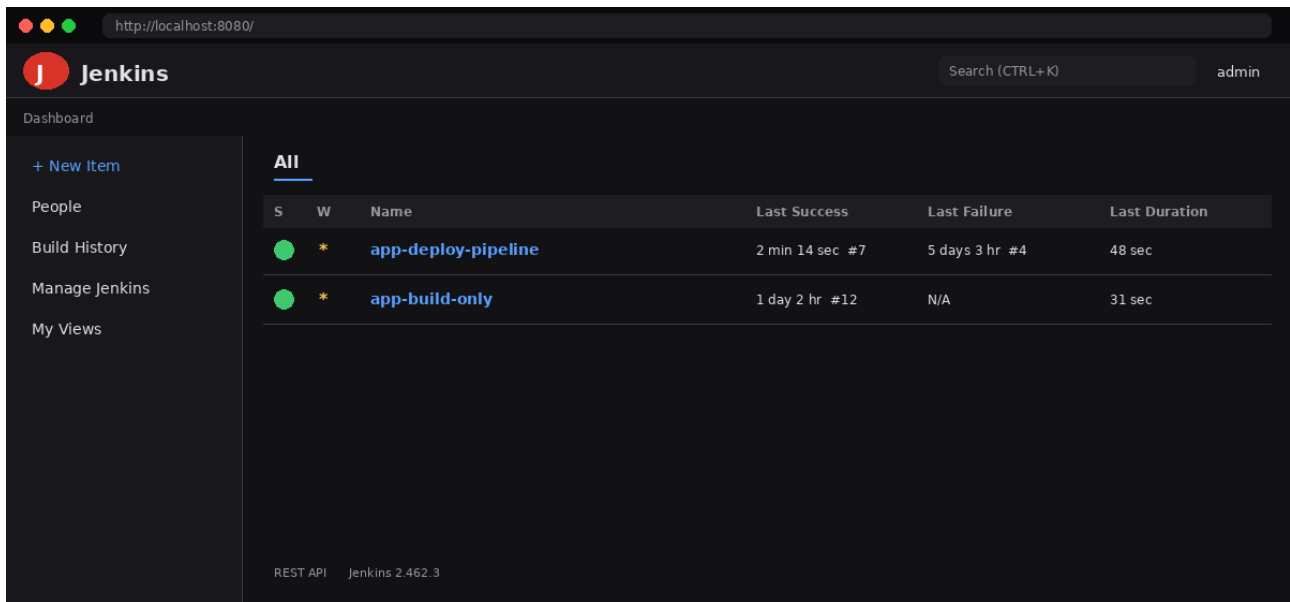


Figure 2: Jenkins dashboard showing the *app-deploy-pipeline* job

Step 3: Prepare the sample application

A small Express application is stored in the GitHub repository **node-deploy-app**. It exposes a **/health** endpoint used later to verify the deployment.

server.js

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send('Hello from Jenkins automated deployment!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'UP', version: process.env.APP_VERSION || 'dev' });
});

if (require.main === module) {
  app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
}
module.exports = app;
```

package.json

```
{
  "name": "node-deploy-app",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "test": "jest"
  },
  "dependencies": { "express": "^4.19.2" },
  "devDependencies": { "jest": "^29.7.0", "supertest": "^7.0.0" }
}
```

server.test.js

```
const request = require('supertest');
const app = require('./server');

test('GET / returns greeting', async () => {
  const res = await request(app).get('/');
  expect(res.statusCode).toBe(200);
});

test('GET /health returns UP', async () => {
  const res = await request(app).get('/health');
  expect(res.body.status).toBe('UP');
});
```

Step 4: Set up SSH access to the deployment server

Jenkins needs passwordless SSH access to the target server. Generate a key as the jenkins user, copy it to the target and add the private key in *Manage Jenkins* > *Credentials* as an **SSH Username with private key** with ID **deploy-server-key**.

ubuntu@jenkins-server: ~ \$

```
sudo -u jenkins ssh-keygen -t ed25519 -f /var/lib/jenkins/.ssh/id_ed25519 -N ""
sudo -u jenkins ssh-copy-id -i /var/lib/jenkins/.ssh/id_ed25519.pub ubuntu@10.0.0.25
sudo -u jenkins ssh ubuntu@10.0.0.25 'echo connected'
```

On the target server (10.0.0.25)

```
sudo apt install -y nodejs npm
sudo npm install -g pm2
sudo mkdir -p /opt/myapp && sudo chown ubuntu:ubuntu /opt/myapp
```

Step 5: Write the Jenkinsfile (deployment steps and post-build actions)

The Jenkinsfile is placed in the repository root. It defines the **triggers**, the six **stages** (checkout, install, test, build, deploy, health check) and the **post** block for post-build actions.

Jenkinsfile

```
pipeline {
  agent any

  environment {
    APP_NAME      = 'myapp'
    DEPLOY_HOST   = '10.0.0.25'
    DEPLOY_USER   = 'ubuntu'
    DEPLOY_DIR    = '/opt/myapp'
    ARTIFACT      = "app-${env.BUILD_NUMBER}.tar.gz"
  }

  triggers {
    githubPush()
    pollSCM('H/5 * * * *')
    cron('H 2 * * *')
  }

  options {
    timestamps()
    timeout(time: 15, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }
}
```

```

stages {
  stage('Checkout') {
    steps {
      git branch: 'main',
        url: 'https://github.com/student-user/node-deploy-app.git'
    }
  }

  stage('Install Dependencies') {
    steps {
      sh 'npm ci'
    }
  }

  stage('Test') {
    steps {
      sh 'npm test'
    }
  }

  stage('Build') {
    steps {
      sh "tar -czf ${ARTIFACT} server.js package.json package-lock.json"
    }
  }

  stage('Deploy') {
    steps {
      sshagent(credentials: ['deploy-server-key']) {
        sh """
          scp -o StrictHostKeyChecking=no ${ARTIFACT} \
            ${DEPLOY_USER}@${DEPLOY_HOST}:${DEPLOY_DIR}/
          ssh -o StrictHostKeyChecking=no ${DEPLOY_USER}@${DEPLOY_HOST} '
            cd ${DEPLOY_DIR} &&
            tar -xzf ${ARTIFACT} &&
            npm ci --omit=dev &&
            APP_VERSION=${BUILD_NUMBER} pm2 restart ${APP_NAME} --update-env || \
            APP_VERSION=${BUILD_NUMBER} pm2 start server.js --name ${APP_NAME}
          '
        """
      }
    }
  }

  stage('Health Check') {
    steps {
      sh "sleep 5 && curl -sf http://${DEPLOY_HOST}:3000/health"
    }
  }
}

post {
  success {
    archiveArtifacts artifacts: 'app-*.tar.gz', fingerprint: true
    mail to: 'team@example.com',
      subject: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
      body: "Deployment succeeded. Details: ${env.BUILD_URL}"
  }
  failure {
    mail to: 'team@example.com',
      subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
      body: "Deployment failed. Check console: ${env.BUILD_URL}console"
  }
  always {
    cleanWs()
  }
}
}

```

Step 6: Create the Pipeline job

- From the dashboard click **New Item**, enter the name **app-deploy-pipeline**, select **Pipeline** and click OK.
- Under **Pipeline**, choose **Pipeline script from SCM**.
- Set SCM to **Git**, Repository URL to **https://github.com/student-user/node-deploy-app.git**, branch to ***/main**.
- Set Script Path to **Jenkinsfile** and click **Save**.

Step 7: Configure build triggers

Under *Configure* > *Build Triggers* enable the three triggers below. They match the triggers block of the Jenkinsfile.

Trigger	Setting	Meaning
GitHub hook trigger for GITScm polling	Enabled	Build on every push
Poll SCM	H/5 * * * *	Check repository every 5 minutes
Build periodically	H 2 * * *	Nightly build around 2 AM

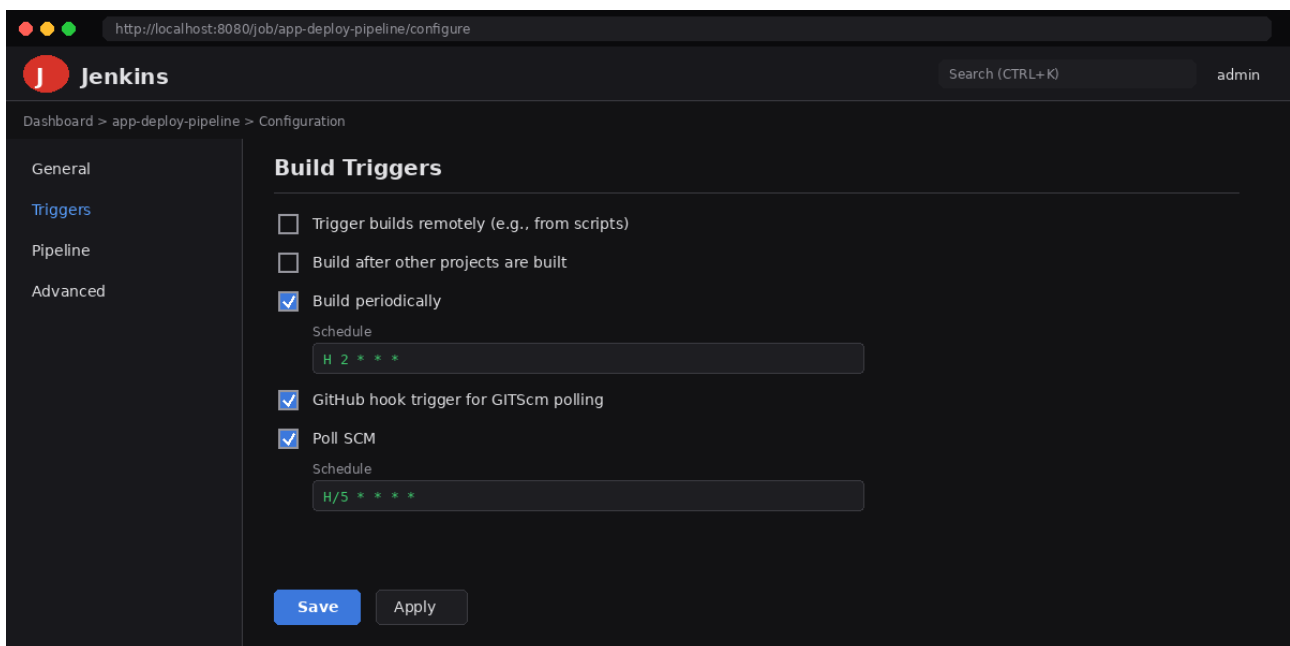


Figure 3: Build Triggers section of the job configuration

Step 8: Add the GitHub webhook

In GitHub open the repository, go to **Settings** > **Webhooks** > **Add webhook** and fill in the fields below. GitHub then notifies Jenkins on every push.

Field	Value
Payload URL	http://<jenkins-public-ip>:8080/github-webhook/
Content type	application/json
Events	Just the push event
Active	Checked

Step 9: Run the pipeline and verify

Commit a change and push it. The webhook starts build **#7** automatically. The Stage View shows each stage completing successfully.

```
student@laptop: ~/node-deploy-app $
```

```
git add .
git commit -m "Update greeting message"
git push origin main
```

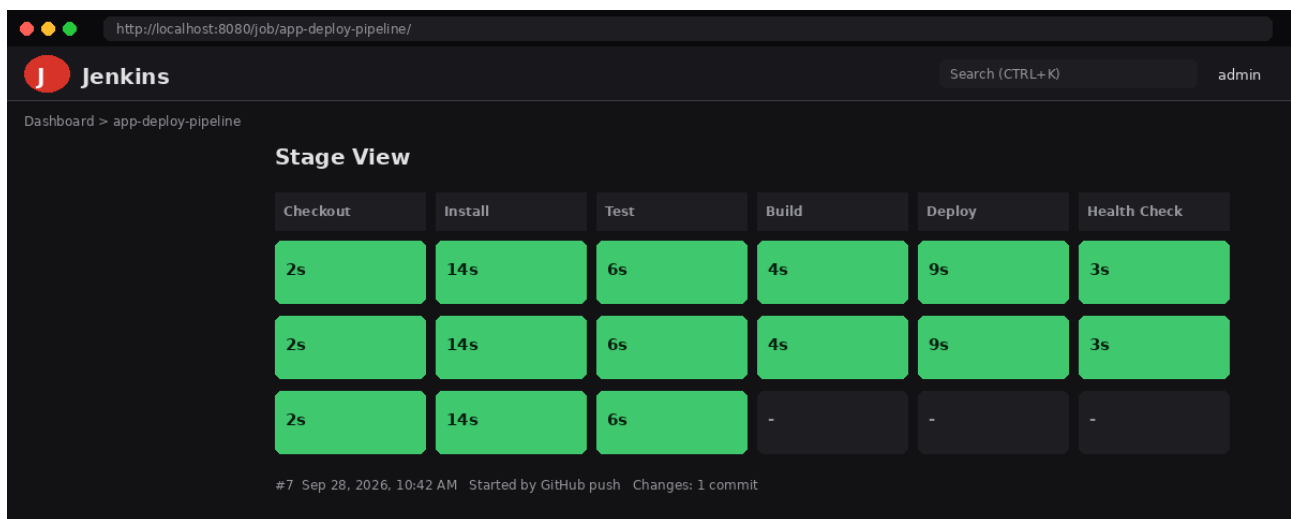


Figure 4: Stage View - every stage of the pipeline passed

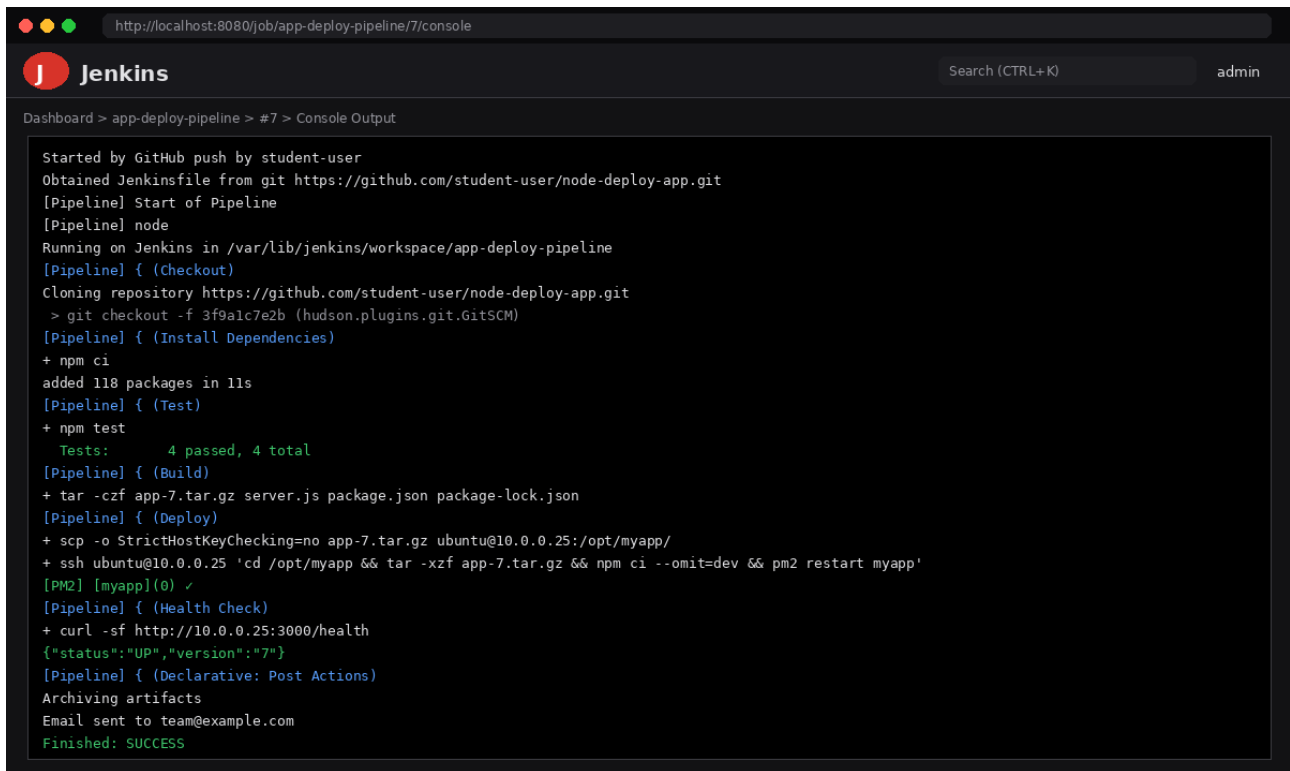
A screenshot of a web browser showing the Jenkins console output for build #7. The browser address bar shows 'http://localhost:8080/job/app-deploy-pipeline/7/console'. The Jenkins logo and 'Jenkins' text are at the top left. A search bar and 'admin' link are at the top right. The console output shows the build process: 'Started by GitHub push by student-user', 'Obtained Jenkinsfile from git https://github.com/student-user/node-deploy-app.git', '[Pipeline] Start of Pipeline', '[Pipeline] node', 'Running on Jenkins in /var/lib/jenkins/workspace/app-deploy-pipeline', '[Pipeline] { (Checkout)', 'Cloning repository https://github.com/student-user/node-deploy-app.git', '> git checkout -f 3f9alc7e2b (hudson.plugins.git.GitSCM)', '[Pipeline] { (Install Dependencies)', '+ npm ci', 'added 118 packages in 11s', '[Pipeline] { (Test)', '+ npm test', 'Tests: 4 passed, 4 total', '[Pipeline] { (Build)', '+ tar -czf app-7.tar.gz server.js package.json package-lock.json', '[Pipeline] { (Deploy)', '+ scp -o StrictHostKeyChecking=no app-7.tar.gz ubuntu@10.0.0.25:/opt/myapp/', '+ ssh ubuntu@10.0.0.25 \'cd /opt/myapp && tar -xzf app-7.tar.gz && npm ci --omit=dev && pm2 restart myapp\'', '[PM2] [myapp](0) ✓', '[Pipeline] { (Health Check)', '+ curl -sf http://10.0.0.25:3000/health', '{"status":"UP","version":"7"}', '[Pipeline] { (Declarative: Post Actions)', 'Archiving artifacts', 'Email sent to team@example.com', 'Finished: SUCCESS'.

Figure 5: Console output of build #7 ending with Finished: SUCCESS

Step 10: Verify the deployed application

student@laptop: ~ \$

```
curl http://10.0.0.25:3000/  
curl http://10.0.0.25:3000/health  
ssh ubuntu@10.0.0.25 'pm2 list'
```

Output

```
Hello from Jenkins automated deployment!  
{"status":"UP","version":"7"}  
  
+---+-----+-----+-----+-----+  
| id | name  | mode  | pid  | ... | status |  
+---+-----+-----+-----+-----+  
| 0  | myapp | fork  | 8841 | ... | online |  
+---+-----+-----+-----+-----+
```

6. Post-Build Actions Summary

Action	Jenkinsfile step	Purpose
Archive artifacts	archiveArtifacts	Keep the deployable .tar.gz of each successful build
E-mail notification	mail (success / failure)	Inform the team about the build result
Workspace cleanup	cleanWs()	Free disk space after every run
Build retention	buildDiscarder	Keep only the last 10 builds

7. Troubleshooting

Problem	Fix
Webhook does not start a build	Ensure port 8080 is reachable from the internet and the URL ends with /github-webhook/
Permission denied (publickey)	Re-run ssh-copy-id for the jenkins user and check the credential ID
npm: command not found	Install Node.js on the Jenkins agent or use the NodeJS plugin
Health check fails	Check pm2 logs myapp on the target and confirm port 3000 is open

8. Result

The application was deployed automatically to the target server whenever code was pushed to GitHub. The pipeline executed checkout, dependency installation, testing, packaging, deployment and a health check, then archived the artifact, sent an e-mail notification and cleaned the workspace.

9. Conclusion

Jenkins Pipeline as Code with build triggers, deployment stages and post-build actions removes manual, error-prone release steps. Each change is tested and deployed consistently, feedback is fast, and the entire process is version-controlled in the Jenkinsfile. The same approach can be extended to Docker images, Kubernetes clusters or cloud platforms.